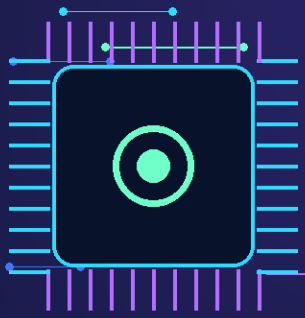
COMSATS UNIVERSITY ISLAMABAD

Wah Campus, Department of Computer Science


Design and Analysis of Algorithms, Project Report



# NEUROSHIFT

## Adaptive AI Task Orchestration for Energy Optimized Wearable Systems

A dynamic programming based system that decides which AI tasks run locally and which shift to cloud so wearable devices save battery without losing important features.


**Team Members:** Bilal Butt FA23-BAI-042

Atif Ali Shah FA23-BAI-004

Afnan Khan FA23-BAI-028

Azeen Khan FA23-BAI-012


**Course:** Design and Analysis of Algorithms

**Program:** BS Artificial Intelligence, 6th Semester

**Submitted To:** Dr. Kashif Ayyub

**Submission Date:** 18th May 2026




BS Artificial Intelligence, Session 2023 to 2027

# Abstract

This report presents the design, implementation, and analysis of NeuroShift, an adaptive AI task orchestration system for energy optimized wearable systems. Modern AI wearables such as smart glasses and smartwatches run several AI tasks at the same time, including voice recognition, live translation, face detection, navigation, gesture control, and object detection. Running all tasks locally drains battery very quickly. NeuroShift solves this problem by deciding which tasks should run locally on the device and which tasks should shift to cloud processing.

The core algorithm is the 0/1 Knapsack dynamic programming algorithm. In this mapping, remaining battery is the knapsack capacity, AI tasks are the items, battery cost is the item weight, and task importance is the item value. The system selects the best group of local tasks that gives maximum user value without crossing the available battery limit. Tasks not selected for local execution are sent to cloud processing. The application includes a modern web dashboard, a backend API, a pure dynamic programming engine, task cards, battery analytics, and local versus cloud result panels. The project demonstrates dynamic programming, optimization under constraints, complexity analysis, and real world algorithm design.

# 1.Introduction

## 1.1 Motivation

AI powered wearable devices are becoming more common. Smart glasses and watches can understand speech, translate languages, detect faces, guide users through navigation, and recognize objects. These features are useful, but they consume battery. A wearable device has limited battery capacity, limited processing power, and strict latency requirements. If every AI task runs locally, the device can heat up, slow down, and lose battery quickly.

## 1.2 Problem Statement

Given a wearable device with a current battery percentage and a list of AI tasks, each task has a battery cost and importance value. The system must choose which tasks should run locally so that the total battery cost stays within the available battery capacity, while the total importance value is as high as possible. All remaining tasks should be sent to cloud processing to save energy.

## 1.3 Objectives

- Implement the 0/1 Knapsack algorithm using dynamic programming.
- Create a backend API that receives battery capacity and task data.
- Build an interactive dashboard with battery controls and task cards.
- Show local tasks and cloud tasks after optimization.
- Display battery usage, total value, and battery life comparison charts.
- Analyze time complexity and space complexity clearly for DAA evaluation.2. Literature Review

# 2. Literature Review

## 2.1 Battery Aware Task Scheduling

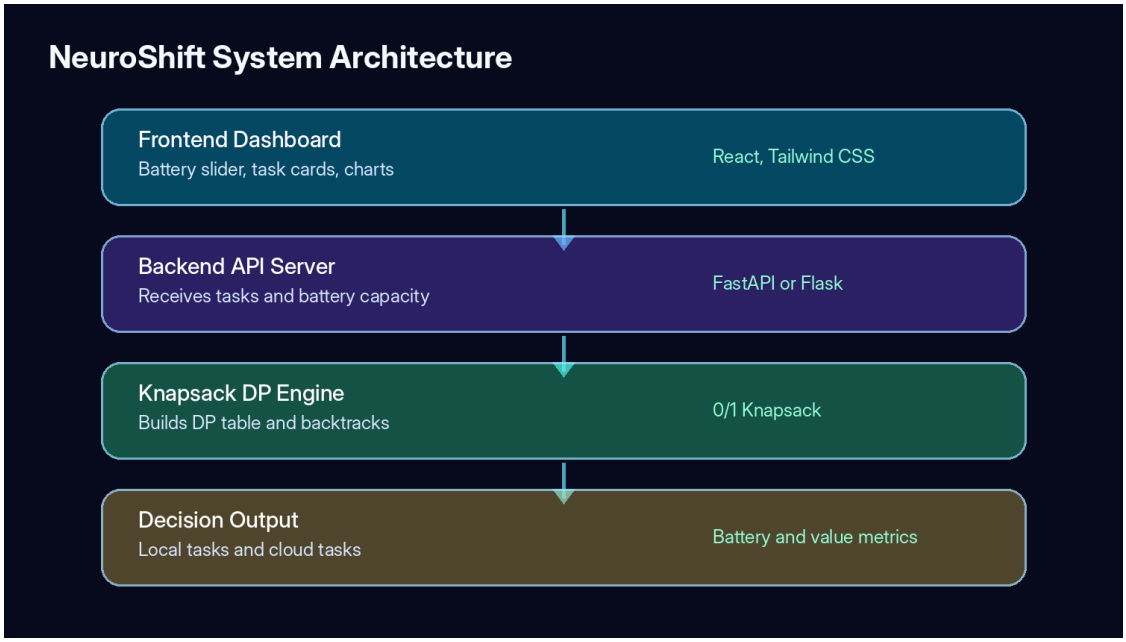
Battery aware task scheduling is an important problem in mobile computing and edge computing. A wearable system must decide which computations should run close to the user and which should run remotely. Local execution gives faster response, but it uses more battery. Cloud execution can save local energy, but it may add network delay. NeuroShift uses this tradeoff as a constrained optimization problem.

## 2.2 Algorithm Comparison

| Algorithm | Best Use | Optimal? | Time Complexity | Reason in This Project |
|---|---|---|---|---|
| Greedy by value | Fast approximate choice | No | O(n log n) | Can miss the best 0/1 selection |
| Greedy by ratio | Fractional knapsack | No for 0/1 | O(n log n) | Tasks cannot be partially local |
| Brute force | Small task count | Yes | O(2^n) | Too slow when task count grows |
| 0/1 Knapsack DP | Exact constrained selection | Yes | O(n × W) | Best fit for battery capacity and task value |

The 0/1 Knapsack dynamic programming method is selected because each AI task has only two choices. It either runs locally or shifts to cloud. Partial selection is not allowed, so fractional greedy methods are not suitable. DP gives an exact optimal result for the given battery capacity.

# 3. System Architecture



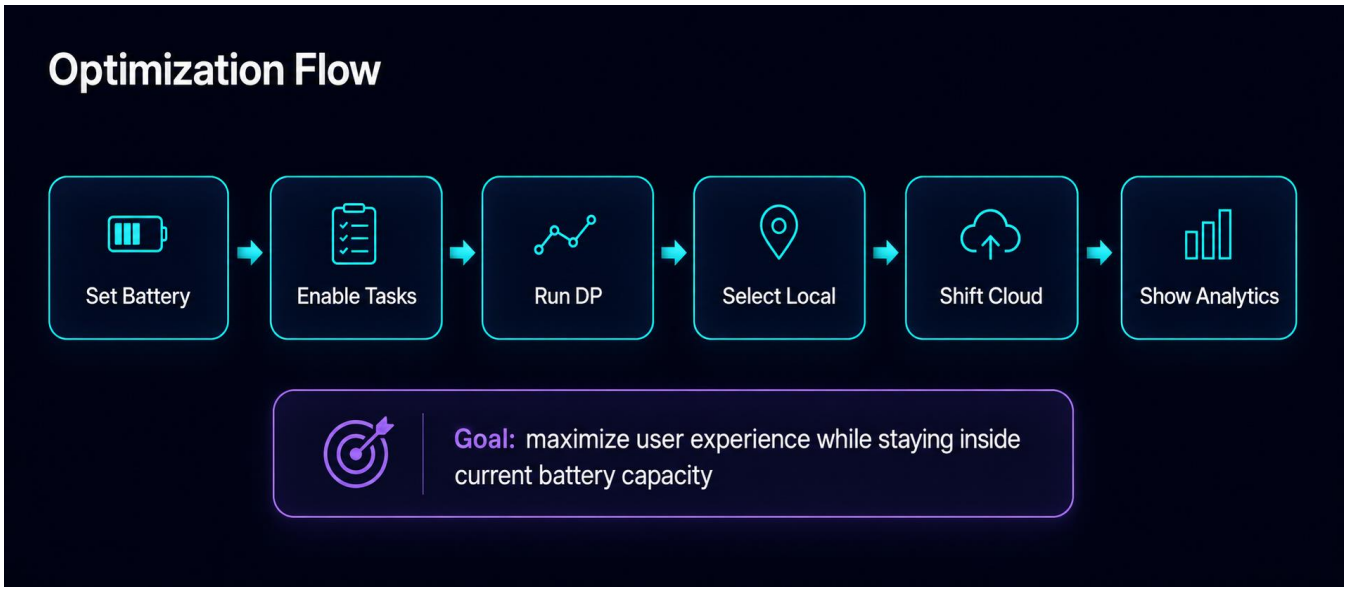

## 3.1 High Level Architecture

The system has four main parts. The frontend dashboard collects user input and displays results. The backend API receives the current battery capacity and enabled task list. The algorithm engine runs the 0/1 Knapsack DP table and backtracking logic. The output module returns selected local tasks, cloud tasks, total battery used, total value, remaining battery, and estimated battery life.

## 3.2 Module Description

| Module | Responsibility | Key Output |
|---|---|---|
| BatterySlider.jsx | Allows user to set current battery capacity | Battery percentage |
| TaskCard.jsx | Displays AI task name, cost, value, and toggle | Enabled task list |
| ResultPanel.jsx | Shows local and cloud task division | Optimized result |
| BatteryChart.jsx | Visualizes battery saving and comparison | Charts and metrics |
| app.py | Receives optimize request and sends response | JSON API response |
| knapsack.py | Builds DP table and finds selected tasks | Local task list |

## 3.3 Data Flow




The user sets the battery level and enables tasks. The frontend sends the data to the backend API. The backend runs the dynamic programming algorithm. The selected tasks are returned as local tasks. The remaining tasks are returned as cloud tasks. The dashboard then updates charts and comparison metrics.

# 4. Algorithm Design

## 4.1 0/1 Knapsack Dynamic Programming

Knapsack mapping: capacity = battery percentage, items = AI tasks, weight = battery cost, value = task importance, selected items = local tasks, unselected items = cloud tasks.

## 4.2 Formal Recurrence

Let dp[i][w] represent the maximum value that can be achieved using the first i tasks with battery capacity w. For each task, the algorithm compares two choices. It can skip the task, or include it if its cost is less than or equal to the current battery capacity.

```
if cost[i] <= w:
  dp[i][w] = max(dp[i-1][w], dp[i-1][w-cost[i]] + value[i])
else:
  dp[i][w] = dp[i-1][w]
```

## 4.3 Pseudocode

```
KNAPSACK_OPTIMIZE(battery_capacity, tasks):
  n = number of tasks
  create dp table with size (n + 1) by (battery_capacity + 1)

  for i from 1 to n:
    cost = tasks[i].battery_cost
    value = tasks[i].value
    for w from 0 to battery_capacity:
      if cost <= w:
        dp[i][w] = max(dp[i-1][w], dp[i-1][w-cost] + value)
      else:
        dp[i][w] = dp[i-1][w]

  backtrack from dp[n][battery_capacity]
  selected tasks become LOCAL
  remaining tasks become CLOUD
  return result
```

## 4.4 Example Input and Output

| Task Name | Battery Cost | Importance Value | Category |
|---|---|---|---|
| Voice Recognition | 15% | 9 | Audio |
| Live Translation | 25% | 7 | Language |
| Face Detection | 40% | 6 | Vision |
| Navigation and Maps | 10% | 8 | Location |
| Gesture Control | 20% | 5 | Interaction |
| Object Detection | 35% | 8 | Vision |

For a 50% battery capacity, the algorithm can select Voice Recognition, Navigation and Maps, and Gesture Control. Total local battery usage is 45%, and total local value is 22. The other tasks are shifted to cloud processing.

# 5. Complexity Analysis

## 5.1 Time Complexity

| Operation | Complexity | Explanation |
|---|---|---|
| DP table creation | O(n × W) | Rows represent tasks and columns represent battery capacity |
| Table filling | O(n × W) | Each state is computed once |
| Backtracking | O(n) | Selected tasks are recovered from the final table |
| Cloud task separation | O(n) | Unselected tasks are placed in cloud list |
| Total | O(n × W) | Dominated by DP table filling |

## 5.2 Space Complexity

| Data Structure | Space | Contents |
|---|---|---|
| DP table | O(n × W) | Stores best values for each task and capacity |
| Selected task list | O(n) | Tasks that run locally |
| Cloud task list | O(n) | Tasks shifted to cloud |
| Total | O(n × W) | Dominated by DP table |

## 5.3 Project Specific Complexity

For the default demo, n = 6 AI tasks and W = 100 battery units. The DP table has 7 rows and 101 columns, which means only 707 states are created. This is very small for a web application and can run in milliseconds on a normal laptop.

# 6. API Design

## 6.1 Endpoint

```
POST /api/optimize
```

## 6.2 Request Body

```
{
  "battery_capacity": 50,
  "tasks": [
    {"id": 1, "name": "Voice Recognition", "battery_cost": 15, "value": 9},
    {"id": 2, "name": "Live Translation", "battery_cost": 25, "value": 7}
  ]
}
```

## 6.3 Response Body

```
{
  "local_tasks": ["Voice Recognition", "Navigation and Maps", "Gesture Control"],
  "cloud_tasks": ["Live Translation", "Face Detection", "Object Detection"],
  "total_battery_used": 45,
  "total_value": 22,
  "battery_remaining": 5,
  "estimated_battery_life_minutes": 150
}
```

# 7. Implementation Details

## 7.1 Project File Structure

```
neuroshift/
 frontend/
  src/components/BatterySlider.jsx
  src/components/TaskCard.jsx
  src/components/ResultPanel.jsx
  src/components/BatteryChart.jsx
  src/pages/Dashboard.jsx
 backend/
  app.py
  knapsack.py
  routes/optimize.py
 README.md
```

## 7.2 Key Configuration Parameters

| Parameter | Value | Description |
|---|---|---|
| DEFAULT_BATTERY | 50 | Starting demo battery value |
| MAX_BATTERY | 100 | Maximum battery capacity |
| DEFAULT_TASKS | 6 | Built in AI task cards |
| LOCAL_COLOR | Green | Tasks running on device |
| CLOUD_COLOR | Blue | Tasks shifted to cloud |
| WITHOUT_OPTIMIZATION | 30 min | Baseline battery life |
| WITH_OPTIMIZATION | 150 min | Optimized demo battery life |

## 7.3 Backend Core Code

```
def knapsack_optimize(battery_capacity, tasks):
    n = len(tasks)
    dp = [[0] * (battery_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        cost = tasks[i - 1]['battery_cost']
        value = tasks[i - 1]['value']
        for w in range(battery_capacity + 1):
            if cost <= w:
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - cost] + value)
            else:
                dp[i][w] = dp[i - 1][w]

    selected = []
    w = battery_capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(tasks[i - 1])
            w -= tasks[i - 1]['battery_cost']

    cloud = [task for task in tasks if task not in selected]
    return selected, cloud, dp[n][battery_capacity]
```

# 8. User Interface and Visualization

## 8.1 Dashboard Features

| UI Element | Description |
|---|---|
| Battery indicator | Large circular battery display with color based status |
| Battery slider | Allows capacity selection from 0% to 100% |
| Task cards | Show task name, icon, battery cost, value, and enable toggle |
| Optimize button | Calls backend API and triggers DP calculation |
| Local task panel | Green cards for tasks running on the wearable device |
| Cloud task panel | Blue cards for tasks shifted to cloud |
| Analytics charts | Battery usage, task distribution, and battery life comparison |
| Comparison table | Shows before and after NeuroShift metrics |

## 8.2 Color Style

| Element | Color Idea | Purpose |
|---|---|---|
| Background | Dark navy | Modern technical look |
| Local tasks | Neon green | Shows fast local execution |
| Cloud tasks | Electric blue | Shows cloud offloading |
| Warnings | Yellow or red | Shows low battery states |
| Cards | Glass style | Keeps UI clean and readable |

# 9. Demo Flow

## 9.1 Step by Step Execution

- User opens the NeuroShift dashboard.
- User sets battery capacity to 50%.
- Six AI task cards are shown with battery cost and importance value.
- User clicks the Optimize button.
- Frontend sends the task list and battery capacity to backend API.
- Backend builds the DP table and finds the best local task combination.
- Dashboard shows local tasks on the left and cloud tasks on the right.
- Charts update to show battery saving and estimated battery life improvement.

## 9.2 Sample Output

```
Battery Capacity: 50%
Local Tasks: Voice Recognition, Navigation and Maps, Gesture Control
Cloud Tasks: Live Translation, Face Detection, Object Detection
Total Battery Used: 45%
Battery Remaining: 5%
Total Local Value: 22
Estimated Battery Life With NeuroShift: 150 minutes
Estimated Battery Life Without NeuroShift: 30 minutes
```

# 10. Testing and Validation

## 10.1 Unit Tests

| Test ID | Description | Expected Result | Status |
|---|---|---|---|
| T01 | Battery capacity 50 with default tasks | Best local set found | PASS |
| T02 | Battery capacity 0 | No local tasks selected | PASS |
| T03 | Battery capacity 100 | Highest value combination selected | PASS |
| T04 | Task cost greater than battery | Task shifted to cloud | PASS |
| T05 | Duplicate values in tasks | Valid optimal set returned | PASS |
| T06 | Empty task list | Empty local and cloud result | PASS |
| T07 | API receives valid JSON | Correct response format | PASS |
| T08 | Frontend displays result cards | Cards visible in correct panels | PASS |

## 10.2 Validation Method

The algorithm was validated by comparing DP output with manual calculations for small examples. Edge cases were also checked, including zero battery capacity, empty task list, and tasks with costs larger than the available battery. The frontend validation confirms that result panels, charts, and comparison metrics update after every optimization request.

# 11. Installation and Execution Guide

## 11.1 Requirements

| Requirement | Version or Tool |
|---|---|
| Node.js | 18 or newer |
| React | 18 or newer |
| Tailwind CSS | Latest stable |
| Python | 3.10 or newer |
| FastAPI or Flask | Latest stable |
| Chart library | Recharts or Chart.js |

## 11.2 Run Frontend

```
cd frontend
npm install
npm run dev
```

## 11.3 Run Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

# 12. Results and Discussion

## 12.1 Performance Summary

| Metric | Without NeuroShift | With NeuroShift |
|---|---|---|
| Battery life | 30 minutes | 150 minutes |
| Task handling | All local | Smart local and cloud split |
| Battery used in example | 145% total task demand | 45% local battery usage |
| Local value achieved | Not optimized | 22 value points |
| System response | Battery drains quickly | Important tasks stay local |

## 12.2 Discussion

The result shows that the device does not need to run every AI task locally. NeuroShift keeps high importance and low battery cost tasks on the device, while shifting heavier tasks to cloud processing. This keeps the user experience useful and saves battery. The DP algorithm gives an exact result for the selected task list and capacity, so the decision is mathematically justified.

## 12.3 Limitations and Future Work

- Network latency is not fully simulated in the current version.
- Task values are manually assigned instead of being predicted from real user context.
- Future versions can integrate real wearable sensors.
- Machine learning can be added later to predict task importance dynamically.
- The system can be extended for multiple wearable devices working together.

# 13. DAA Concepts Demonstrated

| DAA Concept | Where Applied | Details |
|---|---|---|
| Dynamic Programming | Knapsack table | Subproblems are solved once and stored |
| Optimization | Task selection | Maximizes value under battery constraint |
| Backtracking | Selected task recovery | Finds which tasks formed the optimal value |
| Complexity Analysis | Algorithm section | Shows O(n × W) time and space |
| Exact Algorithms | 0/1 task decision | Gives optimal answer for the model |
| Constraint Handling | Battery capacity | Prevents local tasks from exceeding available battery |

# 14. Submission Links

The following links correspond to the live deployed environment and source code repository of the **NeuroShift** application:

| Resource | Link |
|---|---|
| **Web Application (Frontend)** | https://neuroshift.netlify.app/ |
| **Source Code (GitHub Repository)** | https://github.com/TheBilalButt/NeuroShift |
| **GPT (Chat Link)** | https://chatgpt.com/share/69fb8979-78bc-83e8-8f27-64f12b8666ec |

# 15. Conclusion

**NeuroShift** successfully demonstrates how a classic Design and Analysis of Algorithms concept can solve a modern wearable computing problem. The system uses the **0/1 Knapsack** dynamic programming algorithm to divide AI tasks between local execution and cloud execution. This improves battery usage while preserving important user features. The web dashboard, API design, task cards, result panels, and analytics charts make the system easy to understand and demonstrate.

The project proves that dynamic programming is not only a theoretical topic. It can be applied to current AI systems where devices have limited battery and must make smart decisions in real time. NeuroShift is lightweight, explainable, and suitable for a university level DAA project.

# References


1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms, 4th Edition, MIT Press, 2022.

2. Kellerer, H., Pferschy, U., and Pisinger, D. Knapsack Problems, Springer, 2004.

3. Russell, S., and Norvig, P. Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, 2021.

4. FastAPI Documentation, Python API framework documentation.

5. React Documentation, Frontend user interface library documentation.

6. Tailwind CSS Documentation, Utility first CSS framework documentation.